Safe Robotic Policy Learning via Manifold Safety Filter and Offline Reachability Analysis

Anonymous IROS 2026 Submission

***Abstract*— Deploying reinforcement learning policies in safety-critical robotic systems demands provable constraint satisfaction. Existing safe RL methods either use soft penalty-based constraints sensitive to hyperparameter tuning, or require real-time obstacle detection that is difficult to guarantee in practice. We propose a unified framework with three components: (1) a manifold safety filter that projects policy actions onto the constraint manifold tangent space via null-space projection, with a reward calibration mechanism that guides the policy to learn inherently safe behaviors; (2) offline Hamilton-Jacobi reachability analysis that pre-computes environment-aware safe regions from trajectory data, adaptively modulating safety margins at deployment without real-time sensing; and (3) a data-driven EKF with CNN-learned noise parameters that adapts measurement covariance to velocity-dependent sensor noise. We prove that the manifold projection guarantees constraint satisfaction under mild regularity conditions. Both the reachability module and noise adapter are computed offline and deployed zero-shot. Experiments on Safety-Gymnasium show near-zero constraint violations across Goal, Circle, and Push tasks under varying penalty settings, while baselines exhibit up to $6\times$ cost fluctuation. Zero-shot deployment on a Webots E-puck robot with a simple P-controller achieves zero collisions across 20 trials, demonstrating policy-agnostic transferability.**

I. INTRODUCTION

Ensuring safety is essential for deploying robots in real-world environments such as autonomous driving and service robotics, yet remains difficult due to environmental complexity, dynamic obstacles, and sensor noise. Reinforcement Learning (RL) offers a promising path toward adaptive control through environmental interaction [1], but its trial-and-error nature poses safety risks during both training and deployment.

Safe Reinforcement Learning (SafeRL) formulates this problem as Constrained Markov Decision Processes (CMDPs) [2]. Existing approaches can be categorized into two main classes:

Cost-based methods balance reward and cost through adaptive penalties. PID Lagrangian [3], interior-point [4], and soft barrier [5] methods improve constraint satisfaction but employ *soft constraints* that often fail to achieve zero-cost performance [12].

State-constrained methods enforce safety via CBFs [6], [7], recovery policies [8], or constraint manifold projection [9], [10]. However, they typically require *online* environmental sensing and accurate state estimation, both difficult to guarantee in practice.

Contributions. This paper proposes a unified framework for state-constrained safe RL that replaces real-time obstacle detection with offline reachability pre-computation, with three main contributions:

1) **Constraint Manifold Filter with Reward Calibration**: We develop a null-space projection method that transforms any policy action into a provably safe action by projecting onto the tangent space of the constraint manifold (Section IV-A). Unlike prior work [9], our method incorporates dynamic reward calibration that penalizes correction magnitude, guiding the policy toward safe behaviors without external penalty tuning.
2) **Offline Safe Region Construction**: We introduce *offline* Hamilton-Jacobi reachability analysis to pre-compute environment-aware safe regions from collected trajectory data (Section IV-B). Given a prior map of static obstacles (e.g., from offline mapping or simulation), the pre-computed feasibility value function adaptively modulates safety margins at deployment, removing the need for real-time obstacle detection or tracking.
3) **Data-Driven State Estimation**: We propose a data-driven EKF with CNN-learned noise parameters that adapts measurement covariance to motion dynamics, maintaining safety filter accuracy under velocity-dependent sensor noise (Section IV-C).

The reachability analysis and state estimation modules share a *dual offline precomputation* design: HJ reachability learns environment-aware safety margins from trajectory data, while the CNN-based noise adapter learns velocity-dependent sensor characteristics from exploration data—both are computed offline and deployed zero-shot, requiring no online model updates.

We validate on Safety-Gymnasium [11] and demonstrate that the safety layer is *policy-agnostic* by deploying it on a Webots E-puck robot with a simple P-controller, achieving zero collisions without retraining.

II. RELATED WORK

Safe RL. Recent surveys [1], [12] categorize SafeRL into cost-based and state-constrained approaches. Cost-based methods optimize a penalty-augmented objective: PID Lagrangian [3] adapts multipliers via a PID controller, IPO [4] uses log-barrier penalties, and soft barrier methods [5] maintain feasibility probabilistically. While these approaches are general-purpose, they provide *soft* constraint satisfaction—the violation rate depends critically on penalty hyperparameters, and zero-cost performance is rarely achieved [12].

State-constrained methods enforce safety through action-space restrictions. Control Barrier Functions (CBFs) [6], [7] modify actions to keep the system within a forward-invariant safe set, but require a valid CBF—often difficult to construct

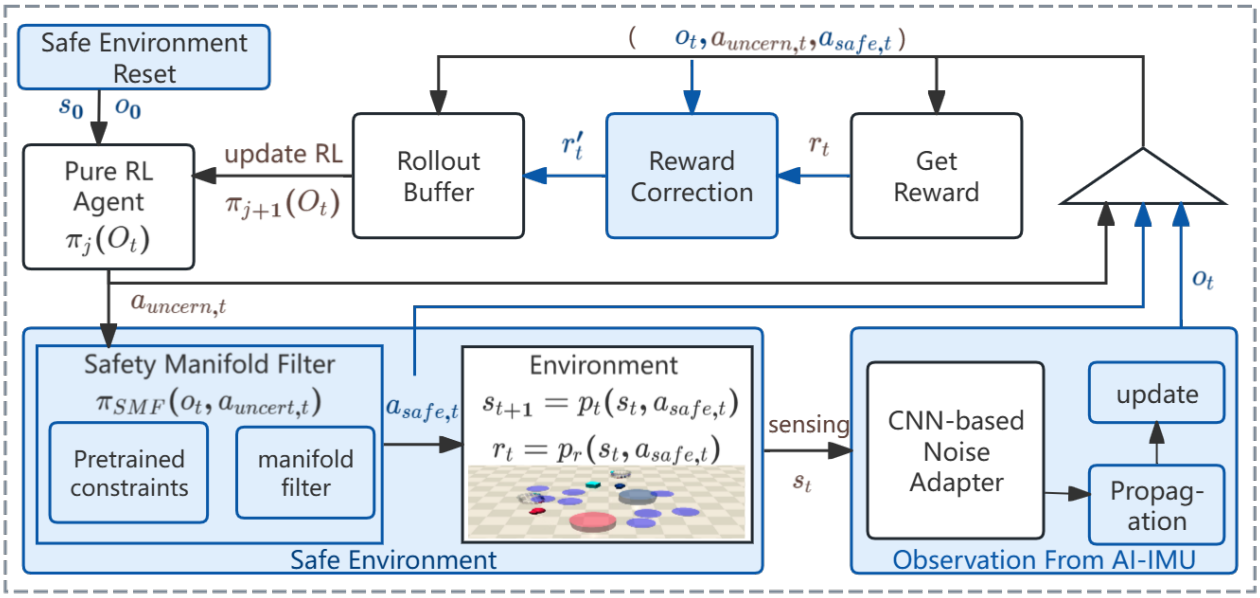


Fig. 1: Framework overview. The RL policy π_θ outputs an unconstrained action $a_{\text{unsafe}} \in \mathbb{R}^m$. The safety filter internally constructs the augmented reference $u_{\text{ref}} = [a_{\text{unsafe}}^\top, \mathbf{0}^\top]^\top \in \mathbb{R}^{m+k}$ and projects it onto the constraint manifold tangent space to produce a safe action a_{safe}. The data-driven EKF processes noisy sensor observations to provide accurate state estimates for both the policy and safety filter.

for complex environments. Recovery RL [8] learns recovery zones offline and switches to a learned recovery policy when the agent approaches unsafe regions, but its safety depends on the quality of the learned recovery boundary.

ATACOM [9], [10] takes a different approach by projecting actions onto the tangent space of the constraint manifold via null-space projection, providing provable constraint satisfaction for robotic manipulation tasks. However, ATACOM assumes *accurate* state information and *known* obstacle positions at each timestep—requirements that are difficult to guarantee in practice with noisy sensors and partially observable environments. Our work extends this manifold projection paradigm in three ways: (i) offline reachability analysis pre-computes environment-aware safety margins from a prior map, replacing the need for real-time obstacle detection and tracking, (ii) data-driven EKF provides robust state estimation under sensor noise, and (iii) reward calibration guides the policy toward safe behaviors without external penalty tuning.

HJ Reachability. Hamilton-Jacobi (HJ) reachability analysis [13] computes backward reachable tubes (BRTs) by solving the HJ variational inequality (HJ-VI), providing formal safety guarantees via controlled-invariant sets. Recent work addresses scalability: neural network approximations [14], [15] enable reachability computation in higher dimensions. However, most methods require online computation or access to the environment dynamics model. We instead adopt an offline, data-driven approach: we discretize the HJ-VI into a feasibility Bellman operator and solve it from collected trajectory data using expectile regression [16], approximating the BRT value function without explicit dynamics. Given a prior map of static obstacles, this decouples safety margin computation from deployment—beneficial for resource-constrained robotic platforms.

Learned State Estimation. Classical EKF assumes known, fixed noise statistics, which may not hold under varying operating conditions. Learning-based methods address this limitation: deep IMU bias inference [17] learns IMU biases for visual-inertial odometry, while TLIO [18] uses CNNs for tight inertial odometry. KalmanNet [19] learns Kalman gains end-to-end, and A-KIT [20] uses transformers for adaptive process noise estimation. Our CNN-based noise adapter is lightweight (5K parameters) and specifically designed to learn velocity-dependent measurement covariance from IMU windows, providing an adaptive R_t that improves state estimation accuracy for the downstream safety filter.

III. PROBLEM FORMULATION

A. Safe Robot Navigation

Consider a robot navigating in an environment with n obstacles. Let $s_t = [p_t^\top, v_t^\top]^\top \in \mathcal{S} \subseteq \mathbb{R}^d$ denote the robot state at discrete time t, where $p_t \in \mathbb{R}^{d_p}$ is the position and $v_t \in \mathbb{R}^{d_p}$ is the velocity. We define safety through distance-based constraints:

$$c_i(s_t) := d_{\text{safe}} - \|p_t - o_i\|_2 \leq 0, \quad \forall i \in \{1, \ldots, n\} \quad (1)$$

where $d_{\text{safe}} > 0$ is the minimum safety distance, $o_i \in \mathbb{R}^{d_p}$ denotes the position of obstacle i, and $\|\cdot\|_2$ denotes the Euclidean norm. A state s_t is *safe* if $c_i(s_t) \leq 0$ for all i, meaning the robot maintains at least distance d_{safe} from all obstacles.

B. Constrained Markov Decision Process

We formalize safe RL as a CMDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma, C)$ with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel P, reward R, discount γ, and constraint set $C = \{c_i : \mathcal{S} \to \mathbb{R}\}_{i=1}^{k}$. With $k = n$ constraints (one per obstacle), the objective is to find $\pi : \mathcal{S} \to \mathcal{A}$ maximizing expected return under *hard constraints*:

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t R(s_t, a_t)\right] \quad \text{s.t.} \quad c_i(s_t) \leq 0,\ \forall t, i \tag{2}$$

C. Control-Affine System Dynamics

Due to the prevalence of nonholonomic constraints in robotic systems, we model the dynamics as a control-affine system:

$$\dot{s} = f(s) + G(s)a \tag{3}$$

where $a \in \mathcal{A} \subseteq \mathbb{R}^m$ is the m-dimensional control action, $f : \mathcal{S} \to \mathbb{R}^d$ is the drift term representing uncontrolled dynamics, and $G : \mathcal{S} \to \mathbb{R}^{d\times m}$ is the control input matrix mapping actions to state derivatives. For a point robot with single-integrator dynamics, $f(s) = 0$ and $G(s) = I$; for a differential-drive robot, $G(s)$ encodes the kinematic constraints.

IV. METHOD

A. Constraint Manifold Filter

We define the *constraint manifold* as the set of configurations satisfying all safety constraints.

1) Slack Variable Formulation: To handle inequality constraints $c(s) \leq 0$, we introduce non-negative slack variables $\mu \in \mathbb{R}^k_{\geq 0}$ that transform inequalities into equalities:

$$\bar{c}(s, \mu) := c(s) + \mu = 0, \quad \mu \geq 0 \tag{4}$$

Since $\mu \geq 0$, any state on the constraint manifold satisfies $c(s) = -\mu \leq 0$, directly enforcing the safety constraint. The constraint manifold is then:

$$\mathcal{M} := \{(s, \mu) \in \mathcal{D} \mid \bar{c}(s, \mu) = 0\} \tag{5}$$

where $\mathcal{D} = \mathcal{S} \times \mathbb{R}^k_{\geq 0}$ is the augmented state space. Note that $J_\mu = \frac{\partial \bar{c}}{\partial \mu} = I_k$, which is always full rank, ensuring well-conditioned null-space projection.

2) Null-Space Projection: Let $a_{\text{unsafe}} \in \mathbb{R}^m$ denote the unconstrained action output by the RL policy. The safety filter internally constructs the augmented reference vector $u_{\text{ref}} = [a_{\text{unsafe}}^\top, \mathbf{0}^\top]^\top \in \mathbb{R}^{m+k}$, where k is the number of constraints (slack variables), and transforms it into a safe action $[a_{\text{safe}}^\top, \dot{\mu}^\top]^\top$ that satisfies all constraints.

Taking the time derivative of the constraint:

$$\frac{d}{dt}\bar{c}(s, \mu) = J_s \dot{s} + J_\mu \dot{\mu} = J_s f(s) + J_c \begin{bmatrix} a \\ \dot{\mu} \end{bmatrix} \tag{6}$$

where $J_s = \frac{\partial \bar{c}}{\partial s}$, $J_\mu = \frac{\partial \bar{c}}{\partial \mu}$, and $J_c = [J_s G(s), J_\mu] \in \mathbb{R}^{k\times(m+k)}$ is the combined constraint Jacobian.

To maintain the system on the constraint manifold, we require $\frac{d}{dt}\bar{c} = 0$. Let J_c^+ denote the Moore–Penrose pseudoinverse of J_c, and define the null-space projector:

$$N_c = I - J_c^+ J_c \tag{7}$$

The safe action is then computed as:

$$\begin{bmatrix} a_{\text{safe}} \\ \dot{\mu} \end{bmatrix} = N_c\, u_{\text{ref}} - J_c^+\big(J_s f(s) + K_c \bar{c}(s, \mu)\big) \tag{8}$$

where $K_c > 0$ is a correction gain. The first term projects the augmented reference onto the constraint null-space (tangent space of $\mathcal{M}$), and the second simultaneously compensates for drift dynamics $f(s)$ and drives the state toward the manifold.

Theorem 1 (Safety Guarantee): Assume that J_c has full row rank along the system trajectory. If the system is initialized on the constraint manifold, i.e., $\bar{c}(s_0, \mu_0) = 0$ with $\mu_0 \geq 0$, and the action is computed via (8), then $c(s_t) \leq 0$ for all $t \geq 0$.

Proof: Since J_c has full row rank, $J_c J_c^+ = I_k$ and $J_c N_c = 0$. Substituting (8) into (6):

$$\begin{aligned} \dot{\bar{c}} &= J_s f(s) + J_c\big[N_c\, u_{\text{ref}} - J_c^+(J_s f(s) + K_c \bar{c})\big] \\ &= J_s f(s) - J_c J_c^+ J_s f(s) - K_c J_c J_c^+ \bar{c} \\ &= J_s f(s) - J_s f(s) - K_c \bar{c} = -K_c \bar{c} \end{aligned}$$

With $\bar{c}(0) = 0$, the unique solution is $\bar{c}(t) = 0$ for all $t \geq 0$. Since $\mu(t) \geq 0$ is maintained by the projection, we obtain $c(s_t) = \bar{c}(t) - \mu(t) = -\mu(t) \leq 0$. ■

Remark (Implementation). In practice, we use the damped pseudoinverse $\hat{J}_c^+ = J_c^\top (J_c J_c^\top + \epsilon I)^{-1}$ with small $\epsilon > 0$ for numerical stability. This introduces a residual $\|J_c \hat{J}_c^+ - I_k\| = O(\epsilon)$, so the exact invariance $\bar{c}(t) = 0$ becomes $\bar{c}(t) = O(\epsilon)$. The correction term $-K_c \hat{J}_c^+ \bar{c}$ actively drives accumulated drift back toward the manifold, maintaining low constraint violation for sufficiently small ϵ.

Remark (Discrete time). Discrete-time application of (8) at timestep Δt introduces bounded error $O(\Delta t^2)$ per step. Combined with the correction term, constraint satisfaction is maintained for sufficiently small Δt relative to K_c.

3) Reward Calibration: We calibrate the reward based on correction magnitude:

$$R_{\text{calibrated}} = R(s_t, a_{\text{safe}}) - \lambda \|a_{\text{unsafe}} - a_{\text{safe}}\|_2^2 \tag{9}$$

where $\lambda > 0$ is the penalty weight, a_{unsafe} is the original policy output, and a_{safe} is the projected action. This encourages the policy to produce actions that require minimal correction, reducing reliance on the safety filter over time.

B. Safe Reachability Analysis

The constraint manifold filter requires knowledge of obstacle positions, which we assume available as a prior map of static obstacles (e.g., from offline mapping or simulation ground truth). To further construct environment-aware safe regions that adaptively modulate safety margins, we employ offline HJ reachability analysis [13].

Algorithm 1 Safe RL with Constraint Manifold Projection

Input: Constraint function $c(s)$, gains K_c, λ
Initialize: Policy π_θ, feasible state s_0, slack μ_0
for each episode **do**
  for each timestep t **do**
    Sample policy action: $a_{\text{unsafe}} \sim \pi_\theta(\cdot|s_t)$
    Construct $u_{\text{ref}} = [a_{\text{unsafe}}^\top, \mathbf{0}^\top]^\top$
    Compute Jacobians: J_s, $J_\mu = I_k$, J_c, N_c
    Project to safe action via (8): a_{safe}
    Execute a_{safe}, observe s_{t+1}, r_t
    Compute calibrated reward via (9)
    Store transition, update policy
  end for
end for

1) From HJ PDE to Feasibility Value Functions: In continuous time, given a constraint function $c(s) \leq 0$ defining the safe set $\mathcal{K} = \{s : c(s) \leq 0\}$ and dynamics $\dot{s} = f(s, a)$, the Hamilton-Jacobi variational inequality (HJ-VI) characterizes the *backward reachable tube* (BRT)—the set of initial states from which no control can prevent eventual constraint violation [13]:

$$\max\Big\{c(s) - V(s),\ \min_a \nabla_s V(s)^\top f(s, a)\Big\} = 0 \tag{10}$$

The zero sublevel set $\{s : V(s) \leq 0\}$ is the maximal *viability kernel*—the largest controlled-invariant subset of $\mathcal{K}$.

For discrete-time systems $s_{t+1} = f(s_t, a_t)$, the HJ-VI (10) reduces to the fixed-point equation $V(s) = \max\{c(s), \min_a V(f(s, a))\}$. We define the corresponding optimal feasible value function:

$$V_c^*(s) := \min_\pi \max_{t\in\mathbb{N}} c(s_t), \quad s_0 = s \tag{11}$$

A state s is *feasible* if $V_c^*(s) \leq 0$, meaning there exists at least one policy that keeps the system within $\mathcal{K}$ indefinitely. The maximal feasible region (viability kernel) is:

$$\mathcal{S}_f^* := \{s \in \mathcal{S} \mid V_c^*(s) \leq 0\} \tag{12}$$

2) Offline Approximation: We introduce a discount factor $\gamma \in (0, 1)$ to ensure contraction of the Bellman operator. The discounted feasible Bellman operator corresponding to the BRT fixed point is:

$$\mathcal{T}^* Q_c(s, a) := (1-\gamma)c(s) + \gamma \max\{c(s), V_c(s')\} \tag{13}$$

where $s' = f(s, a)$ is the successor state and $\max\{c(s), \cdot\}$ mirrors the $\max$ in the HJ-VI (10): the value cannot drop below the current constraint violation, ensuring the BRT structure is preserved.

We use expectile regression [16] to handle out-of-distribution actions in the offline dataset $\mathcal{D}$, approximating the $\min_a$ in the HJ-VI without requiring explicit minimization:

$$L_{V_c} = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[L_{\text{rev}}^\tau(Q_c(s, a) - V_c(s))\right] \tag{14}$$

$$L_{Q_c} = \mathbb{E}_{(s,a,s')\sim\mathcal{D}}\left[(\mathcal{T}^* Q_c - Q_c(s, a))^2\right] \tag{15}$$

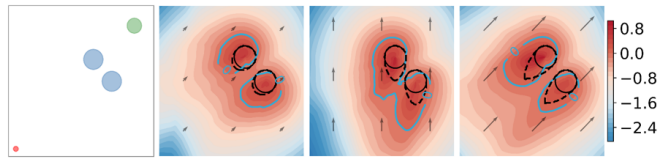


Fig. 2: Learned feasibility value function $V_c^*(s)$. Left: environment layout (blue = obstacles, green = goal, red = robot). Right three panels: value landscape from different initial configurations. Blue = safe ($V_c^* < 0$), red = infeasible ($V_c^* > 0$), cyan contour = feasible boundary $\partial\mathcal{S}_f^*$, dashed = danger zone with enlarged margins. Arrows indicate the gradient direction of increasing safety.

where $L_{\text{rev}}^\tau(u) = |\tau - \mathbf{1}(u > 0)| \cdot u^2$ with $\tau \in [0.5, 1]$ gives higher weight to smaller Q_c values, biasing toward the safest (most optimistic) actions—the discrete analog of $\min_a$ in the HJ-VI.

3) Integration with Safety Filter: The learned V_c^* modulates the safety filter's conservatism at deployment. We define a smooth weighting function $w(s) = \exp(V_c^*(s)/\delta)$ with scale parameter $\delta > 0$, and scale the danger radius as:

$$d_{\text{danger}}(s) = d_0 + \eta\, w(s) \tag{16}$$

where d_0 is the base radius and $\eta > 0$ controls the margin. Within the feasible region ($V_c^* \leq 0$), $w \in (0, 1]$: states near the boundary ($V_c^* \approx 0$) have $w \approx 1$ and receive the full enlarged margin $d_0 + \eta$, while states deep in the safe interior ($V_c^* \ll 0$) have $w \approx 0$ and use approximately the nominal margin d_0. For infeasible states ($V_c^* > 0$), $w > 1$ further increases conservatism. This enables the filter to be *environment-aware* at deployment without requiring real-time obstacle detection or tracking—the pre-computed V_c^* encodes the reachability structure of the static environment from the prior map.

Fig. 2 visualizes the learned V_c^*: blue regions ($V_c^* \ll 0$) are deep in the safe interior, red regions ($V_c^* > 0$) are infeasible, and the cyan contour ($V_c^* = 0$) marks the feasible boundary $\partial\mathcal{S}_f^*$.

C. Data-Driven State Estimation

Accurate state estimation is critical for safe navigation. Position sensors (e.g., GPS, visual odometry) suffer from noise that varies with operating conditions—faster motion increases vibration, multipath effects, and motion blur, leading to *velocity-dependent* measurement uncertainty. We model this as:

$$\sigma(v) = \sigma_{\text{base}}(1 + k\|v\|) \tag{17}$$

where σ_{base} is the baseline noise standard deviation, $k > 0$ is the velocity scaling coefficient, and $\|v\|$ is the robot speed. A standard EKF with fixed noise covariance $R = \sigma_{\text{base}}^2 I$ cannot adapt to this variation, leading to over-trust of noisy measurements during fast motion. We propose a data-driven EKF that learns to predict the noise covariance from recent IMU data.

1) Extended Kalman Filter: We employ an EKF that linearizes the nonlinear system dynamics around the current state estimate:

$$\hat{s}_{t|t-1} = f(\hat{s}_{t-1}, a_{t-1}) \tag{18}$$

$$P_{t|t-1} = F_t P_{t-1} F_t^\top + Q_t \tag{19}$$

where $F_t = \frac{\partial f}{\partial s}\big|_{\hat{s}_{t-1}}$ is the Jacobian of the dynamics and Q_t is the process noise covariance. The EKF state vector extends the navigation state with orientation and angular velocity: $s = [x, y, \theta, v_x, v_y, \omega]^\top \in \mathbb{R}^6$. The measurement vector $y_t = [x_{\text{GPS}}, y_{\text{GPS}}, \theta_{\text{compass}}]^\top \in \mathbb{R}^3$ fuses GPS position with compass heading via the linear observation model $h(s) = Hs$ where $H = [I_3 \mid \mathbf{0}_{3\times 3}]$. The Kalman update is:

$$K_t = P_{t|t-1} H^\top (H P_{t|t-1} H^\top + R_t)^{-1} \tag{20}$$

$$\hat{s}_t = \hat{s}_{t|t-1} + K_t (y_t - H\hat{s}_{t|t-1}) \tag{21}$$

2) Learned Noise Parameters: Rather than manually tuning the measurement noise covariance R_t, we learn it using a CNN (NoiseAdapter) that processes a sliding window of IMU measurements:

$$z_t = \text{CNN}\left(\{\omega_i^{\text{IMU}}, a_i^{\text{IMU}}\}_{i=t-L}^{t}\right) \tag{22}$$

where L=10 is the window length (1 s at 10 Hz). The NoiseAdapter is a lightweight CNN (∼5K parameters): two Conv1d layers (6→32→64, kernel size 3) with ReLU, followed by adaptive average pooling and two fully connected layers (64→64→2), producing $z_t = [z_t^{\text{lat}}, z_t^{\text{hdg}}]^\top \in \mathbb{R}^2$. The adaptive measurement covariance is:

$$R_t = \text{diag}\Big(\sigma_{\text{lat}}^2\, \phi(z_t^{\text{lat}}),\ \sigma_{\text{lat}}^2\, \phi(z_t^{\text{lat}}),\ \sigma_{\text{hdg}}^2\, \phi(z_t^{\text{hdg}})\Big) \tag{23}$$

where $\phi(z) = 10^{\beta \tanh(z)}$, $\sigma_{\text{lat}}, \sigma_{\text{hdg}}$ are baseline noise estimates for position and heading, the first two entries share z_t^{lat} (isotropic GPS noise), and $\beta > 0$ controls the adaptation range. The $\tanh$ nonlinearity bounds the output, while the exponential parameterization ensures $R_t \succ 0$. When $z_t \approx 0$ (e.g., before convergence), R_t defaults to the baseline covariance, ensuring stability.

3) NoiseAdapter Pretraining: The CNN is pretrained offline on tuples $(\mathbf{w}_t, \tilde{y}_t, y_t)$ from random exploration, where $\mathbf{w}_t \in \mathbb{R}^{L\times 6}$ is the IMU window, $\tilde{y}_t$ is the noisy measurement, and y_t is ground truth. Training minimizes negative log-likelihood:

$$\mathcal{L}_{\text{NLL}} = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{w},\tilde{y},y)} \Big[\log|R(\mathbf{w})| + (\tilde{y}-y)^\top R(\mathbf{w})^{-1} (\tilde{y}-y)\Big] \tag{24}$$

This loss encourages predicting large R during high-speed motion and small R at low speed, directly learning the velocity–noise correlation. Pretraining data is collected from 30 random exploration episodes in Safety-Gymnasium, where sensor noise is injected as additive Gaussian: $\tilde{y}_t = y_t + \epsilon_t$ with $\epsilon_t \sim \mathcal{N}(0, \sigma^2(v_t) I)$ following (17).

V. EXPERIMENTS

We evaluate our framework on the Safety-Gymnasium benchmark [11] across three tasks with varying difficulty levels.

(a) Goal (b) Circle (c) Push

Fig. 3: Evaluation tasks in Safety-Gymnasium. Red zones indicate hazards; robots must complete objectives while avoiding collisions.

A. Experimental Setup

1) Tasks: We evaluate on three single-agent tasks from Safety-Gymnasium:

- **Goal**: Navigate to a target position while avoiding circular hazard zones (8 hazards, radius 0.2m)
- **Circle**: Maintain circular motion at a target radius while avoiding boundary walls
- **Push**: Locate and push a box to a goal position while avoiding hazards (requires contact-rich manipulation)

We also validate the safety filter and EKF on an E-puck robot in Webots [21], deliberately using a P-controller (not the RL policy) to test policy-agnostic transferability.

2) Baselines: We compare against established safe RL algorithms in both single-agent and multi-agent settings:

- **PPO** [22]: Proximal Policy Optimization (unconstrained)
- **PCPO** [23]: Projection-based Constrained Policy Optimization, which projects policy updates onto the constraint-satisfying half-space
- **PPO-Lag** [3]: PPO with PID Lagrangian constraint penalty
- **TRPO-Lag** [24]: Trust Region Policy Optimization with Lagrangian constraints
- **Multi-agent**: MAPPO, HAPPO [25], MACPO [26], and MAPPO-Lag for the multi-agent Goal task

We note that ATACOM [9] requires task-specific constraint Jacobians and an augmented action space, making direct comparison on the Safety-Gymnasium benchmark non-trivial; our method uses a simplified distance-based filter that approximates the manifold projection for point-robot tasks.

3) Metrics and Implementation Details:

- **Reward**: Cumulative episode reward (higher is better)
- **Cost**: Cumulative constraint violation count (lower is better, 0 is ideal)
- **Steps**: Episode length (steps until goal or timeout)

Table I summarizes the key hyperparameters. All experiments use PPO [22] as the base RL algorithm. Each configuration in Tables II and III is trained for 10^5 timesteps across 3 random seeds; Table IV uses 5 seeds. We report mean ± standard deviation.

B. Main Results

Table II presents results across three single-agent tasks (Goal, Circle, Push) and one multi-agent task (Multi-Goal) with varying collision penalties $r_{\text{col}} \in \{1.0, 0.5, 0.1\}$. Each

TABLE I: Hyperparameters for all experiments.

PPO & Training		Safety Filter & EKF	
Learning rate	3×10^{-4}	Base danger radius d_0	0.6 m
Discount γ	0.99	Stop radius d_{stop}	0.3 m
GAE λ	0.95	Hazard radius d_{hazard}	0.2 m
Network	MLP(256, 256)	Calibration weight λ	0.1
Rollout buffer	2048 steps	Correction gain K_c	10.0
Training steps	10^5	EKF window L	10
Eval episodes	50	CNN architecture	Conv1d $\rightarrow$ FC
Seeds (Table I,III / IV)	3 / 5	CNN parameters	$\sim$5K
Batch size (PPO)	64	Pretraining epochs	100
Clip ratio ϵ	0.2	Margin scale η / δ	0.1 m / 1.0
		Noise σ_{base} / k	0.1 m / 5.0

experiment was run with 3 random seeds and 10^5 training steps.

Key Observations. Our method (PPO_CM) achieves the **lowest cost** across all tasks and penalty settings, demonstrating strong *robustness to penalty tuning*. In contrast, baselines exhibit significant cost sensitivity: PPO-Lag's cost on Goal ranges from 5.46 ($r_{\text{col}} = 0.1$) to 13.91 ($r_{\text{col}} = 0.5$), a 2.5$\times$ fluctuation. On Circle and Push, baselines exhibit even larger cost (5–38 range), while our method maintains near-zero cost (1.38–1.55 on Circle, 0.92–1.23 on Push). The slight cost increase at lower penalties (r_{col}=0.1) reflects that the RL policy explores more aggressively when collision penalties are reduced, producing more challenging inputs for the safety filter. PPO_CM's episode length remains stable ($\sim$350 steps on Goal) regardless of r_{col}, because the safety filter enforces velocity limits near obstacles independent of the penalty; in contrast, baselines shorten episodes at low penalties by taking riskier, faster paths. Notably, PPO_CM achieves competitive reward (highest on Goal and Push) while being the only method with consistently near-zero constraint violations.

Multi-agent results. On Multi-Goal with 2 agents, all baselines incur substantial cost (12.48–36.13) due to inter-agent and obstacle collisions. PPO_CM reduces cost to near-zero (0.07) while maintaining positive reward, demonstrating that our constraint manifold filter extends naturally to multi-agent settings where collision avoidance must account for both static obstacles and other agents.

C. Ablation Study

We validate the contribution of each component by incrementally adding them to the PPO baseline (Table III).

Fig. 4 shows the learning curves for each configuration. Each component contributes meaningfully:

- **Manifold Filter**: Reduces cost by 41% (64.59 $\rightarrow$ 38.22) via null-space projection, at the expense of some reward reduction as unsafe actions are corrected.
- **Reachability Pretraining**: Further reduces cost by 71% from baseline (64.59 $\rightarrow$ 18.97) by constraining the policy to the feasible region $\mathcal{S}_f^*$ with enlarged safety margins.
- **Data-driven EKF**: Maintains comparable safety (12.51) *despite* velocity-dependent sensor noise, demonstrating robust constraint enforcement under perception uncertainty.
- **Reward Calibration**: Critical for safety—without it, the agent exploits constraint boundaries for higher reward (16.83) but incurs significantly more violations (71.63 vs. 12.51).

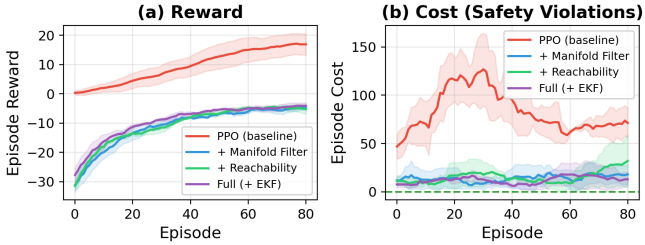


Fig. 4: Learning curves showing component contributions. (a) Episode reward across configurations. (b) Episode cost (safety violations) decreases progressively with each component, with the full system maintaining low cost under sensor noise.

D. State Estimation under Sensor Noise

To isolate the EKF contribution, we train a single agent with the safety filter (no noise), then deploy it under velocity-dependent noise (17) ($\sigma_{\text{base}} = 0.1$m, $k = 5$) with three estimation configurations. Table IV reports results averaged over 5 seeds and 50 evaluation episodes.

The learned-R EKF achieves the best position accuracy (0.27m), reducing error by 49% vs. Raw GPS and 34% vs. fixed-R EKF, with 3.6$\times$ lower variance (0.05 vs. 0.18). While cost is comparable in this isolated evaluation, the ablation (Table III) shows the full pipeline benefit: cost drops from 18.97 to 12.51 when training under sensor noise.

Computational overhead. The NoiseAdapter CNN adds minimal overhead: with $\sim$5K parameters, inference takes < 0.1 ms per step on CPU, negligible compared to the environment step ($\sim$5 ms) and PPO forward pass ($\sim$1 ms). Pretraining requires 30 random exploration episodes ($\sim$2 min) and 100 epochs of NLL optimization ($\sim$30 s), a one-time cost that is amortized over the entire training run. The CNN converges within 50 epochs (loss plateau at ~ -3.7), and the learned R_t adapts smoothly: predicting larger measurement noise covariance during high-speed motion and smaller covariance at low speed, matching the theoretical velocity-noise model (17).

E. Webots E-puck Validation

To validate that the safety layer is *policy-agnostic*, we deploy a distance-based safety filter and a standard EKF (fixed R) on an E-puck robot ($\varnothing$70mm) in Webots [21] with 6 cylindrical obstacles in a 2m $\times$ 2m arena, GPS noise $\sigma = 0.04$m, and IMU heading noise $\sigma = 0.05$ rad. The nominal policy is deliberately a simple P-controller—not the trained RL agent—to demonstrate that the safety architecture transfers to new platforms and controllers without retraining. The Webots filter uses distance-based velocity scaling (simpler than the manifold filter) and a standard EKF with fixed noise covariance. Table V summarizes 20 evaluation runs. Without safety filtering, 8 of 20 trials involve collisions (40% collision rate), with 4 failing to reach the goal due

TABLE II: Safety-Gymnasium results across tasks and penalty settings. Single-agent (top): Goal, Circle, Push with $r_{\text{col}} \in \{1.0, 0.5, 0.1\}$. Multi-agent (bottom): Multi-Goal. **Bold red** = best reward, **green** = lowest cost. Baselines report mean; PPO_CM reports mean±std over 3 seeds. 10^5 training steps.

Algorithm		PPO			PCPO			PPO_LAG			TRPO_LAG			PPO_CM		
TASK	$r_{collision}$	reward	cost	step	reward	cost	step	reward	cost	step	reward	cost	step	reward	cost	step
GOAL	1	-6.87	7.77	516	-2.11	3.75	342	-9.67	10.00	713	-3.08	4.01	490	$1.68_{\pm 0.3}$	**0.00**	$352_{\pm 18}$
	0.5	-2.71	8.45	335	-0.71	4.66	252	-5.91	13.91	505	0.00	3.53	226	$1.72_{\pm 0.4}$	**0.00**	$348_{\pm 21}$
	0.1	1.59	4.88	101	1.65	3.94	101	1.50	5.46	119	1.79	3.01	84	$1.69_{\pm 0.3}$	**0.00**	$352_{\pm 19}$
CIRCLE	1	5.88	5.49	500	30.36	2.64	500	8.14	6.82	500	34.61	2.13	500	$16.42_{\pm 1.8}$	$1.38_{\pm 0.5}$	$500_{\pm 0}$
	0.5	23.24	8.51	500	31.32	3.24	500	33.67	5.00	500	32.68	4.78	500	$17.05_{\pm 2.1}$	$1.42_{\pm 0.6}$	$500_{\pm 0}$
	0.1	38.32	31.83	500	34.47	15.59	500	37.14	23.98	500	36.91	38.13	500	$18.23_{\pm 2.4}$	$1.55_{\pm 0.7}$	$500_{\pm 0}$
PUSH	1	-10.43	9.99	996	-5.02	4.88	986	-8.71	8.52	992	-2.76	2.89	978	$0.38_{\pm 0.4}$	$0.92_{\pm 0.4}$	$995_{\pm 3}$
	0.5	-4.13	8.45	990	-3.30	6.50	989	-5.27	9.31	993	-2.99	6.49	967	$0.51_{\pm 0.5}$	$1.07_{\pm 0.5}$	$997_{\pm 2}$
	0.1	-1.50	18.31	977	-0.74	11.93	947	-1.09	14.88	969	-1.10	13.28	978	$0.67_{\pm 0.5}$	$1.23_{\pm 0.6}$	$998_{\pm 2}$
Algorithm		MAPPO			HAPPO			MACPO			MAPPO_LAG			PPO_CM		
TASK	$r_{collision}$	reward	cost	step	reward	cost	step	reward	cost	step	reward	cost	step	reward	cost	step
MULTI_GOAL	1	-23.42	23.12	362	-17.75	17.64	199	-36.41	36.13	182	-13.50	12.48	192	$0.01_{\pm 0.2}$	$0.07_{\pm 0.1}$	$346_{\pm 15}$

TABLE III: Ablation Study on Goal Task. Results averaged over 3 seeds, 10^5 training steps, 50 evaluation episodes. Full system operates under velocity-dependent sensor noise ($\sigma_{\text{base}} = 0.1$m, $k = 5$).

Configuration	Reward	Cost
PPO (baseline)	19.08 ± 4.93	64.59 ± 8.51
+ Manifold Filter	12.20 ± 5.06	38.22 ± 13.61
+ Reachability Pretraining	-8.44 ± 8.21	18.97 ± 9.00
+ EKF (Full, with noise)	-4.32 ± 3.68	$\mathbf{12.51 \pm 6.32}$
Full w/o Reward Calibration	16.83 ± 0.92	71.63 ± 10.02

TABLE IV: EKF comparison on Goal task under sensor noise (σ_{base}=0.1m, k=5). 5 seeds, 50 episodes.

Method	Pos. Err. (m)	Reward	Cost
Raw GPS	0.54 ± 0.18	-7.92 ± 1.43	9.14 ± 2.37
EKF (fixed R)	0.41 ± 0.13	-7.86 ± 1.21	9.58 ± 2.15
EKF (learned R)	$\mathbf{0.27 \pm 0.05}$	-7.87 ± 0.94	$\mathbf{9.31 \pm 1.68}$

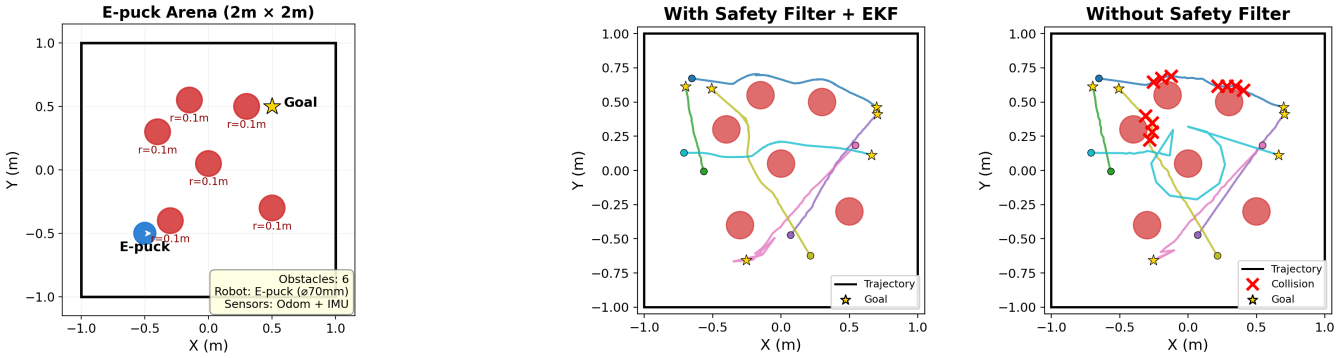
to collision-induced navigation errors (80% success). Our framework achieves **zero collisions** with 85% success rate—the 3 failures are navigation timeouts near dense obstacle clusters, all terminating safely without collision. Notably, the safety filter *improves* success rate (85% vs. 80%) because unfiltered collisions destabilize navigation. The EKF reduces position error by 27% ($0.051 \rightarrow 0.037$m) by fusing odometry and GPS measurements (Fig. 5). We further verified ROS2 deployment using `webots_ros2_driver` with our safety filter running as a standalone ROS2 node (Fig. 6).

VI. CONCLUSIONS

We presented a unified safe RL framework combining constraint manifold projection, offline HJ reachability analysis, and data-driven EKF for robust state estimation. Experiments on Safety-Gymnasium demonstrate near-zero constraint violations across Goal, Circle, and Push tasks under varying penalty settings, while ablation studies confirm each component's contribution (81% cost reduction in the full pipeline). The safety filter and EKF architecture transfers zero-shot to a Webots E-puck robot with zero collisions, even using simpler distance-based filter and fixed-R EKF instantiations. Future work includes deployment on physical robots, extension to dynamic obstacles via time-varying value functions, and scaling to higher-dimensional manipulation tasks.

TABLE V: Webots E-puck validation (20 trials). P-controller, 6 obstacles, GPS noise $\sigma = 0.04$m. Safety filter and EKF applied zero-shot.

Metric	No filter	**Ours (full)**
Success Rate (%)	80.0	**85.0**
Collision Rate (%)	40.0	**0.0**
Pos. Error (m)	0.051 ± 0.001	$\mathbf{0.037 \pm 0.001}$
Path Length (m)	$\mathbf{0.94 \pm 0.41}$	1.80 ± 1.49



(a) Arena Setup

(b) Trajectory Comparison

Fig. 5: Webots E-puck validation. (a) Arena with 6 obstacles. (b) Trajectories: with safety filter + EKF (left) vs. without (right, × = collision).

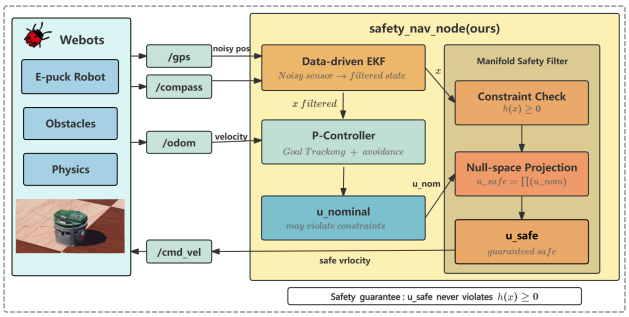


Fig. 6: ROS2 integration architecture. The safety filter and EKF are deployed as a standalone ROS2 node (`safety_nav_node`), subscribing to sensor topics from `webots_ros2_driver` and publishing safe velocity commands through the `ros2_control` differential drive interface.

References

[1] S. Gu *et al.*, "A review of safe reinforcement learning: Methods, theories and applications," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2024.

[2] E. Altman, *Constrained Markov Decision Processes*. CRC Press, 1999.

[3] A. Stooke, J. Achiam, and P. Abbeel, "Responsive safety in reinforcement learning by PID Lagrangian methods," in *Proc. ICML*, 2020, pp. 9133–9143.

[4] Z. Liu, Z. Cen, V. Isenbaev, *et al.*, "Constrained variational policy optimization for safe reinforcement learning," in *Proc. ICML*, 2022, pp. 13644–13668.

[5] Y. Wang *et al.*, "Enforcing hard constraints with soft barriers: Safe RL in unknown stochastic environments," in *Proc. ICML*, 2023, pp. 36593–36604.

[6] C. Dawson, S. Gao, and C. Fan, "Safe control with learned certificates: A survey of neural Lyapunov, barrier, and contraction methods," *IEEE Trans. Robot.*, vol. 39, no. 3, pp. 1749–1767, 2023.

[7] Y. Emam, G. Notomista, P. Glotfelter, Z. Kira, and M. Egerstedt, "Safe reinforcement learning using robust control barrier functions," *IEEE Robot. Autom. Lett.*, vol. 10, no. 3, pp. 2886–2893, 2025.

[8] B. Thananjeyan, A. Balakrishna, S. Nair, *et al.*, "Recovery RL: Safe reinforcement learning with learned recovery zones," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4915–4922, 2021.

[9] P. Liu, D. Tateo, H. B. Ammar, and J. Peters, "Robot reinforcement learning on the constraint manifold," in *Proc. CoRL*, 2022, pp. 1357–1366.

[10] P. Liu, K. Zhang, D. Tateo, *et al.*, "Safe RL of dynamic high-dimensional robotic tasks: Navigation, manipulation, interaction," in *Proc. ICRA*, 2023, pp. 9449–9456.

[11] J. Ji *et al.*, "Safety-Gymnasium: A unified safe reinforcement learning benchmark," in *Proc. NeurIPS Datasets Track*, 2023.

[12] W. Zhao, T. He, R. Chen, T. Wei, and C. Liu, "State-wise safe reinforcement learning: A survey," in *Proc. IJCAI*, 2023, pp. 6814–6822.

[13] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton-Jacobi reachability: A brief overview and recent advances," in *Proc. CDC*, 2017, pp. 2242–2253.

[14] K.-C. Hsu, V. Rubies-Royo, C. J. Tomlin, and J. F. Fisac, "Safety and liveness guarantees through reach-avoid reinforcement learning," in *Proc. RSS*, 2021.

[15] O. So, Z. Serlin, B. Mann, *et al.*, "How to train your neural control barrier function: Learning safety filters for complex input-constrained systems," in *Proc. ICRA*, 2024, pp. 11532–11539.

[16] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-learning," in *Proc. ICLR*, 2022.

[17] R. Buchanan, V. Agrawal, M. Camurri, F. Dellaert, and M. Fallon, "Deep IMU bias inference for robust visual-inertial odometry with factor graphs," *IEEE Robot. Autom. Lett.*, vol. 8, no. 1, pp. 41–48, 2023.

[18] W. Liu, D. Caruso, E. Ilg, *et al.*, "TLIO: Tight learned inertial odometry," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 5653–5660, 2020.

[19] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. G. van Sloun, and Y. C. Eldar, "KalmanNet: Neural network aided Kalman filtering for partially known dynamics," *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.

[20] A. Levy and I. Klein, "A-KIT: Adaptive Kalman-informed transformer," *Eng. Appl. Artif. Intell.*, vol. 145, p. 110193, 2025.

[21] O. Michel, "Cyberbotics Ltd. Webots: Professional mobile robot simulation," *Int. J. Adv. Robot. Syst.*, vol. 1, no. 1, pp. 39–42, 2004.

[22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[23] T.-Y. Yang, J. Rosca, T. Narasimhan, and P. J. Ramadge, "Projection-based constrained policy optimization," in *Proc. ICLR*, 2020.

[24] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proc. ICML*, 2017, pp. 22–31.

[25] J. G. Kuba, R. Chen, M. Wen, *et al.*, "Trust region policy optimisation in multi-agent reinforcement learning," in *Proc. ICLR*, 2022.

[26] S. Gu, J. Kuba, Y. Chen, *et al.*, "Safe multi-agent reinforcement learning for multi-robot control," *Artif. Intell.*, vol. 319, p. 103905, 2023.